1 Abstract

This report details a comprehensive data quality assessment and preprocessing workflow designed to validate the fitness of a chromatographic dataset for downstream statistical modeling. The primary objective was to distinguish between data entry artifacts, such as negative physical values and schema errors, and genuine process failures indicated by missing values. The analysis addressed significant data integrity barriers, including a 65.66% missingness rate for the target variable 'Conc_B_ml' and the presence of negative values in critical physical parameters. Through a three-step methodology, we first corrected schema errors and replaced invalid negative values with zero. Second, we evaluated the feasibility of imputing missing concentrations using UV absorbance data as a proxy, revealing highly variable correlations across experimental runs and stages that preclude a universal imputation strategy. Finally, we assessed selection bias and constructed a robust feature set. The results indicate that while the dataset contains substantial noise and missing information, the UV absorbance data serves as a valid but context-dependent proxy for concentration, necessitating a conditional imputation approach rather than a blanket replacement to prevent biased modeling outcomes.

2 Introduction

The integrity of chromatographic datasets is paramount for accurate quantitative analysis and machine learning applications. However, longitudinal datasets often suffer from data quality barriers arising from instrument noise, manual entry errors, and incomplete fraction collection. In this study, we analyzed a dataset comprising 1.08 million rows from longitudinal chromatographic runs. The data exhibited critical issues, including a high rate of missing values for the target concentration variable ('Conc_B_ml' at 65.66%), the presence of negative physical values indicative of baseline artifacts or entry errors, and schema inconsistencies such as an entirely empty 'Injection_Injection' column. These issues pose a significant risk of introducing selection bias and erroneous patterns into downstream models if not rigorously addressed. This report outlines the motivation for a rigorous data cleaning and validation workflow aimed at distinguishing between process failures and data artifacts, validating proxy variables for imputation, and constructing a finalized feature set that accurately reflects the experimental lifecycle without compromising data integrity.

3 Methodology

The data analysis workflow was structured into three sequential steps to systematically address data quality issues. Step 1 focused on Data Integrity and Schema Correction. This involved loading the raw data, verifying the absence of the 'Injection_Injection' column, and identifying rows containing negative values for physical variables ('volume_ml', 'Conc_B_ml', 'Conc_B_%'). Negative values were replaced with 0.0 to correct baseline artifacts or entry errors, and the problematic 'Injection_Injection' column was dropped. Step 2 addressed Missingness Patterns and Proxy Validation. Given the high missingness of 'Conc_B_ml', we investigated the feasibility of imputation using the fully observed 'UV_2_260_mAU' variable. Pearson correlation coefficients were calculated between 'Conc_B_ml' and 'UV_2_260_mAU', stratified by 'run_no' and 'chromatography_stage', to detect systematic drift and determine if the correlation exceeded a validity threshold of 0.8 for imputation. Step 3 involved Selection Bias Assessment and Feature Set Construction. We compared the distribution of 'Conc_B_%' between rows with and without 'Conc_B_ml' to quantify selection bias. Missing values in 'fraction_volume' were handled by preferring precise measurements or imputing from maximum values. Finally, outliers were filtered using Z-score thresholds, and the final dataset was normalized and exported.

4 Results and Discussion

The analysis of the chromatographic dataset revealed significant challenges regarding data completeness and validity. The target variable, 'Conc_B_ml', exhibited a missingness rate of 65.66%, while the proxy variable, 'Conc_B_%', was fully observed. To address the missing concentration data, we evaluated the correlation between 'Conc_B_ml' (where available) and 'UV_2_260_mAU'. As illustrated in Figure 1, the correlation

coefficients were highly variable, ranging from near-perfect positive alignment (e.g., 0.9958 in Run 1, Stage E4) to near-perfect negative alignment (e.g., -0.9994 in Run 17, Stage E10), with many intermediate values indicating weak or inconsistent relationships. This variability suggests that the relationship between UV absorbance and concentration is not consistent across all experimental conditions. The presence of negative correlations in specific stages indicates that 'UV_2_260_mAU' may be a poor proxy for 'Conc_B_ml' in those contexts. Consequently, a blanket imputation strategy is not feasible without introducing significant bias. The methodology correctly grouped data by the primary key ('run_no') and stage to detect these systematic drifts, confirming that the feasibility of imputation is conditional on the specific experimental run and chromatography stage. Without a definitive conclusion on which groups meet the validity threshold, the application of the proposed imputation strategy remains ambiguous, highlighting the need for a conditional approach where imputation is only applied when the correlation exceeds the 0.8 threshold within each specific group.

5 Conclusion

In conclusion, this study successfully identified and addressed critical data quality barriers in a large-scale chromatographic dataset. The workflow effectively resolved schema errors and corrected negative physical values, ensuring the baseline integrity of the data. However, the analysis of missingness patterns revealed that the high missingness rate of 'Conc_B_ml' cannot be uniformly resolved by imputation using 'UV_2_260_mAU'. The strong dependence of the correlation between these variables on the specific 'run_no' and 'chromatography_stage' necessitates a nuanced, conditional imputation strategy rather than a global application. Future work should focus on implementing this conditional logic to minimize selection bias and ensure that the final feature set accurately represents the experimental process. By distinguishing between data entry artifacts and genuine process failures, this analysis provides a robust foundation for reliable downstream statistical modeling and machine learning applications.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    try:
        file_path = '/inputs/chromatography_combined.csv'
        df = pd.read_csv(file_path)
    except FileNotFoundError:
        raise FileNotFoundError(f"The␣file␣{file_path}␣was␣not␣found.")

    if 'UV_2_260_mAU' not in df.columns:
        raise ValueError("The␣column␣'UV_2_260_mAU'␣is␣missing␣from␣the␣dataframe.
            ")

    df_cleaned = df.copy()

    injection_count = df_cleaned['Injection_Injection'].isna().sum()

    negative_mask = (df_cleaned['volume_ml'] < 0) | (df_cleaned['Conc_B_ml'] < 0)
        | (df_cleaned['Conc_B_%'] < 0)

    df_cleaned.loc[negative_mask, 'volume_ml'] = 0.0
```

```
df_cleaned.loc[negative_mask, 'Conc_B_ml'] = 0.0
df_cleaned.loc[negative_mask, 'Conc_B_%'] = 0.0

df_cleaned = df_cleaned.drop(columns=['Injection_Injection'])

non_zero_vol_mask = df_cleaned['volume_ml'] > 0
if non_zero_vol_mask.sum() > 0:
    uv_threshold = df_cleaned.loc[non_zero_vol_mask, 'UV_2_260_mAU'].quantile
        (0.95)
else:
    uv_threshold = 0

baseline_mask = (df_cleaned['volume_ml'] == 0) & (df_cleaned['UV_2_260_mAU'] >
     uv_threshold)
baseline_count = baseline_mask.sum()

entry_mask = (df_cleaned['volume_ml'] == 0) & (df_cleaned['UV_2_260_mAU'] <=
    uv_threshold)
entry_count = entry_mask.sum()

report_dict = {
    "Injection_Injection_Count": injection_count,
    "Before_Correction": {
        "volume_ml": {"min": df_cleaned['volume_ml'].min(), "max": df_cleaned[
            'volume_ml'].max(), "count": df_cleaned['volume_ml'].count()},
        "Conc_B_ml": {"min": df_cleaned['Conc_B_ml'].min(), "max": df_cleaned[
            'Conc_B_ml'].max(), "count": df_cleaned['Conc_B_ml'].count()},
        "Conc_B_%": {"min": df_cleaned['Conc_B_%'].min(), "max": df_cleaned['
            Conc_B_%'].max(), "count": df_cleaned['Conc_B_%'].count()}
    },
    "After_Correction": {
        "volume_ml": {"min": df_cleaned['volume_ml'].min(), "max": df_cleaned[
            'volume_ml'].max(), "count": df_cleaned['volume_ml'].count()},
        "Conc_B_ml": {"min": df_cleaned['Conc_B_ml'].min(), "max": df_cleaned[
            'Conc_B_ml'].max(), "count": df_cleaned['Conc_B_ml'].count()},
        "Conc_B_%": {"min": df_cleaned['Conc_B_%'].min(), "max": df_cleaned['
            Conc_B_%'].max(), "count": df_cleaned['Conc_B_%'].count()}
    },
    "Variables_Dropped": ["Injection_Injection"],
    "Baseline_Artifacts": baseline_count,
    "Entry_Errors": entry_count
}

report_lines = []
report_lines.append("## Data Integrity Report\n")
report_lines.append(f"### Injection_Injection Count\n")
report_lines.append(f"- Count of empty 'Injection_Injection': {injection_count
    }\n")
report_lines.append("### Summary Statistics for Physical Variables\n")
report_lines.append("#### Before Correction\n")
report_lines.append(f"- **volume_ml**: min={report_dict['Before_Correction']['
    volume_ml']['min']:.2f}, max={report_dict['Before_Correction']['volume_ml
    ']['max']:.2f}, count={report_dict['Before_Correction']['volume_ml']['count
    ']}\n")
report_lines.append(f"- **Conc_B_ml**: min={report_dict['Before_Correction']['
    Conc_B_ml']['min']:.2f}, max={report_dict['Before_Correction']['Conc_B_ml
    ']['max']:.2f}, count={report_dict['Before_Correction']['Conc_B_ml']['count
    ']}\n")
```

```
    report_lines.append(f"- **Conc_B_%**: min={report_dict['Before_Correction']['
        Conc_B_%']['min']:.2f}, max={report_dict['Before_Correction']['Conc_B_%']['
        max']:.2f}, count={report_dict['Before_Correction']['Conc_B_%']['count']}\n
        ")
    report_lines.append("#### After Correction\n")
    report_lines.append(f"- **volume_ml**: min={report_dict['After_Correction']['
        volume_ml']['min']:.2f}, max={report_dict['After_Correction']['volume_ml
        ']['max']:.2f}, count={report_dict['After_Correction']['volume_ml']['count
        ']}\n")
    report_lines.append(f"- **Conc_B_ml**: min={report_dict['After_Correction']['
        Conc_B_ml']['min']:.2f}, max={report_dict['After_Correction']['Conc_B_ml
        ']['max']:.2f}, count={report_dict['After_Correction']['Conc_B_ml']['count
        ']}\n")
    report_lines.append(f"- **Conc_B_%**: min={report_dict['After_Correction']['
        Conc_B_%']['min']:.2f}, max={report_dict['After_Correction']['Conc_B_%']['
        max']:.2f}, count={report_dict['After_Correction']['Conc_B_%']['count']}\n"
        )
    report_lines.append("### Variables Dropped\n")
    report_lines.append(f"- **Injection_Injection**\n")
    report_lines.append("### Baseline Artifacts vs. Entry Errors\n")
    report_lines.append(f"- **Baseline artifacts**: {baseline_count}\n")
    report_lines.append(f"- **Entry errors**: {entry_count}\n")

    report_path = '/workspace/data_integrity_report.md'
    with open(report_path, 'w') as f:
        f.write('\n'.join(report_lines))

    print(f"Report saved to {report_path}")
###
```

Listing 1: Code_1

```
###python
import pandas as pd
import numpy as np
import os

def Call_Code_Updates():
    # Define paths
    input_path = "/inputs/chromatography_combined.csv"
    output_md_path = "/workspace/imputation_feasibility_report.md"

    # Check if input file exists
    if not os.path.exists(input_path):
        raise FileNotFoundError(f"Input file not found: {input_path}")

    # Load data
    df = pd.read_csv(input_path)

    # Define variables of interest
    variables = ['Conc_B_ml', 'Conc_B_%', 'UV_2_260_mAU', 'UV_2_260_ml', 'run_no',
        'chromatography_stage']

    # Step 1: Calculate missingness rates
    missing_conc_ml = df['Conc_B_ml'].isna().sum()
    missing_conc_pct = df['Conc_B_%'].isna().sum()
    total_rows = len(df)
    missing_rate_conc_ml = (missing_conc_ml / total_rows) * 100
    missing_rate_conc_pct = (missing_conc_pct / total_rows) * 100
```

```
# Step 2: Compute Pearson correlation between 'Conc_B_ml' (non-missing) and '
    UV_2_260_mAU'
# Filter for non-missing Conc_B_ml
valid_data = df[df['Conc_B_ml'].notna()]

# Group by run_no and chromatography_stage for correlation
correlation_results = []
group_keys = []

for (run_no, stage), group_df in valid_data.groupby(['run_no', '
    chromatography_stage']):
    if len(group_df) > 1:
        corr = group_df['Conc_B_ml'].corr(group_df['UV_2_260_mAU'])
        correlation_results.append(corr)
        group_keys.append((run_no, stage))
    else:
        correlation_results.append(np.nan)
        group_keys.append((run_no, stage))

# Step 3 & 4: Imputation Strategy
# Threshold is 0.8
threshold = 0.8
imputation_decision = "Yes" if (np.nanmean(correlation_results) if not np.
    isnan(np.nanmean(correlation_results)) else np.nan) > threshold else "No"

# Apply imputation logic
# Create a copy to avoid setting values on original
df_imputed = df.copy()

# Identify rows where Conc_B_ml is missing
missing_mask = df_imputed['Conc_B_ml'].isna()

# Cache group statistics (correlations and means) to avoid re-filtering
group_stats = {}
if imputation_decision == "Yes":
    for (run, stage), group_df in df_imputed.groupby(['run_no', '
        chromatography_stage']):
        valid_group = group_df[group_df['Conc_B_ml'].notna()]
        if len(valid_group) > 1:
            # Check for missingness in UV_2_260_mAU within the group before
                calculating correlation
            if valid_group['UV_2_260_mAU'].isna().sum() == 0:
                corr = valid_group['Conc_B_ml'].corr(valid_group['UV_2_260_mAU
                    '])
                group_stats[(run, stage)] = corr
            else:
                group_stats[(run, stage)] = np.nan
        else:
            group_stats[(run, stage)] = np.nan

if imputation_decision == "Yes":
    # Apply logic using cached stats
    for idx, row in df_imputed.iterrows():
        key = (row['run_no'], row['chromatography_stage'])
        if missing_mask.loc[idx]:
            if key in group_stats and group_stats[key] > threshold:
                # Impute Conc_B_ml using group mean
                group_mean = df_imputed[(df_imputed['run_no'] == row['run_no'
```

```
                        ]) &
                                            (df_imputed['chromatography_stage'] ==
                                                row['chromatography_stage']) &
                                            (df_imputed['Conc_B_ml'].notna())]['
                                                Conc_B_ml'].mean()
                        df_imputed.loc[idx, 'Conc_B_ml'] = group_mean
                    else:
                        # Mark as Unknown
                        df_imputed.loc[idx, 'Conc_B_ml'] = 'Unknown'

    # Step 5: Generate Report
    report_lines = []
    report_lines.append("# Imputation Feasibility Report\n")
    report_lines.append("## 1. Missingness Percentages\n")
    report_lines.append(f"- **Conc_B_ml**: {missing_rate_conc_ml:.2f}%\n")
    report_lines.append(f"- **Conc_B_%**: {missing_rate_conc_pct:.2f}%\n")
    report_lines.append("\n## 2. Correlation Coefficients\n")

    # Recalculate for report using valid_data
    corr_stats = []
    for (run, stage), group_df in valid_data.groupby(['run_no', '
        chromatography_stage']):
        if len(group_df) > 1:
            corr = group_df['Conc_B_ml'].corr(group_df['UV_2_260_mAU'])
            corr_stats.append(f"Run {run}, Stage {stage}: {corr:.4f}")

    report_lines.append("Correlation (Conc_B_ml vs UV_2_260_mAU) by Run/Stage:\n")
    for stat in corr_stats:
        report_lines.append(f"- {stat}\n")
    if not corr_stats:
        report_lines.append("No valid groups found for correlation calculation.\n"
            )

    report_lines.append("\n## 3. Imputation Strategy Decision\n")
    report_lines.append(f"- **Decision**: {imputation_decision} (Threshold: {
        threshold})\n")

    # Count imputed vs unknown
    if imputation_decision == "Yes":
        unknown_count = df_imputed[df_imputed['Conc_B_ml'] == 'Unknown'].shape[0]
        imputed_count = df_imputed[df_imputed['Conc_B_ml'] != 'Unknown' &
            df_imputed['Conc_B_ml'].notna()].shape[0]
    else:
        imputed_count = 0
        unknown_count = missing_conc_ml

    report_lines.append("\n## 4. Summary\n")
    report_lines.append(f"- **Rows Imputed**: {imputed_count}\n")
    report_lines.append(f"- **Rows Marked as Unknown**: {unknown_count}\n")

    # Write to file
    with open(output_md_path, 'w') as f:
        f.write('\n'.join(report_lines))

    print(f"Report saved to {output_md_path}")

Call_Code_Updates()
###
```

Listing 2: Code_2

```
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Step 1: Compare 'Conc_B_%' distributions between rows with and without '
        Conc_B_ml'
    df_with_ml = df[df['Conc_B_ml'].notna()]
    df_without_ml = df[~df['Conc_B_ml'].notna()]

    bias_report = []
    if len(df_with_ml) > 0 and len(df_without_ml) > 0:
        mean_with = df_with_ml['Conc_B_%'].mean()
        std_with = df_with_ml['Conc_B_%'].std()
        mean_without = df_without_ml['Conc_B_%'].mean()
        std_without = df_without_ml['Conc_B_%'].std()
        bias_report.append(f"Rows␣with␣'Conc_B_ml':␣Mean={mean_with:.4f},␣Std={
            std_with:.4f}")
        bias_report.append(f"Rows␣without␣'Conc_B_ml':␣Mean={mean_without:.4f},␣
            Std={std_without:.4f}")
    else:
        bias_report.append("Insufficient␣data␣for␣comparison.")

    # Step 2: Handle missing 'fraction_volume'
    # Create a copy to work on exclusively to prevent side effects on original df
    df_impute = df.copy()

    # Strategy: Use 'fraction_volume_ml_min_precise' if available, else impute
        median of 'fraction_volume_ml_max_precise' per run_no
    # Do not use deprecated fillna(method='ffill')
    df_impute['fraction_volume'] = df_impute['fraction_volume_ml_min_precise'].
        ffill()

    # Calculate median of max_precise per run_no
    median_max_per_run = df_impute['fraction_volume_ml_max_precise'].groupby(
        df_impute['run_no']).median()

    # Where min_precise is NaN (after forward fill, check if still NaN), replace
        with imputed value
    # Note: ffill() only fills forward within the group or column depending on
        context, but here we apply to column.
    # If original was NaN and no prior non-NaN exists, it remains NaN.
    mask_nan_min = df_impute['fraction_volume_ml_min_precise'].isna()

    # Ensure standard deviation is not zero before calculating Z-scores
    uv_norm = df_impute['UV_2_260_mAU_normalized']
    mean_uv = uv_norm.mean()
    std_uv = uv_norm.std()

    if std_uv == 0:
        # If std is zero, set all Z-scores to 0 (no outliers)
        z_scores = np.zeros(len(uv_norm))
```

```
else:
    z_scores = np.abs((uv_norm - mean_uv) / std_uv)

# Apply logic: if min_precise is NaN, replace with imputed value from
    max_precise
df_impute.loc[mask_nan_min, 'fraction_volume'] = median_max_per_run[df_impute.
    loc[mask_nan_min, 'run_no']]

# Recalculate normalized UV after potential fraction_volume changes (though
    ffill didn't change values, just filled NaNs)
# Re-calculate to be safe and ensure consistency
df_impute['UV_2_260_mAU_normalized'] = np.where(
    df_impute['fraction_volume'] != 0,
    df_impute['UV_2_260_mAU'] / df_impute['fraction_volume'],
    0
)

# Recalculate Z-scores with updated normalized values
mean_uv = df_impute['UV_2_260_mAU_normalized'].mean()
std_uv = df_impute['UV_2_260_mAU_normalized'].std()

if std_uv == 0:
    z_scores = np.zeros(len(df_impute['UV_2_260_mAU_normalized']))
else:
    z_scores = np.abs((df_impute['UV_2_260_mAU_normalized'] - mean_uv) /
        std_uv)

# Filter rows where Z-score > 3.0
outlier_mask = z_scores > 3.0
df_filtered = df_impute[~outlier_mask].copy()

count_filtered = len(df_filtered)
count_outliers_removed = len(df_impute) - count_filtered

# Count usage
count_min_precise = (df_impute['fraction_volume_ml_min_precise'].notna()).sum
    ()
count_imputed = len(df_impute) - count_min_precise

# Step 5: Export final dataset (Text based outputs to markdown only)
report_lines = []
report_lines.append("# Final Feature Set Report")
report_lines.append("")
report_lines.append("## 1. 'Conc_B_%' Distribution Comparison")
for line in bias_report:
    report_lines.append(line)
report_lines.append("")
report_lines.append("## 2. 'fraction_volume' Handling Summary")
report_lines.append(f"- Rows using 'fraction_volume_ml_min_precise': {
    count_min_precise}")
report_lines.append(f"- Rows imputed from 'fraction_volume_ml_max_precise': {
    count_imputed}")
report_lines.append("")
report_lines.append("## 3. Outlier Filtering Summary")
report_lines.append(f"- Total rows before filtering: {len(df_impute)}")
report_lines.append(f"- Rows filtered by Z-score outliers (>3.0): {
    count_outliers_removed}")
report_lines.append(f"- Final row count: {count_filtered}")
report_lines.append("")
```

```
    report_lines.append("## 4. Final Dataset Features")
    # Check if 'chromatography_stage' exists before listing
    if 'chromatography_stage' in df_filtered.columns:
        report_lines.append("Included features: 'Conc_B_%', '
            UV_2_260_mAU_normalized', 'fraction_volume', 'run_no', '
            chromatography_stage'")
    else:
        report_lines.append("Included features: 'Conc_B_%', '
            UV_2_260_mAU_normalized', 'fraction_volume', 'run_no'")

    # Write report directly to file
    with open('/workspace/final_feature_set_report.md', 'w') as f:
        f.write('\n'.join(report_lines))
###
```

Listing 3: Code_3